ПРУФ-ИТ / PROOFIT

Наблюдаемость и эксплуатационный контроль

Prometheus, Grafana и Alertmanager: метрики, приватность, диагностические контракты и воспроизводимая приёмка.

Редакция документа: 11 октября 2026. Архитектурные утверждения связаны с указанными модулями; числовые результаты — с протоколами измерений, их датами и областью применимости. Обновление документа не означает повторного проведения всех экспериментов.

Роли: специалист, работодатель, явно одобренный эксперт. Окончательное решение о найме, трактовка спорного evidence и разрешение на обработку данных не передаются автономному score.

Навигация

Наблюдаемость и эксплуатационный контроль **1**

Навигация . . . 2

1. Эксплуатационная задача и границы наблюдения . . . 3

2. Архитектура и потоки наблюдений . . . 3

3. Каталог сигналов и измерительные контракты . . . 4

4. Формулы и различение смыслов . . . 4

5. Сигналы инцидента и операционные действия . . . 4

6. Безопасность, приватность и минимизация . . . 5

7. Ресурсные бюджеты и жизненный цикл . . . 5

8. Запуск и воспроизводимая проверка . . . 6

9. Фактическая приёмка текущего контура . . . 6

10. Первичные источники и привязка к реализации . . . 6

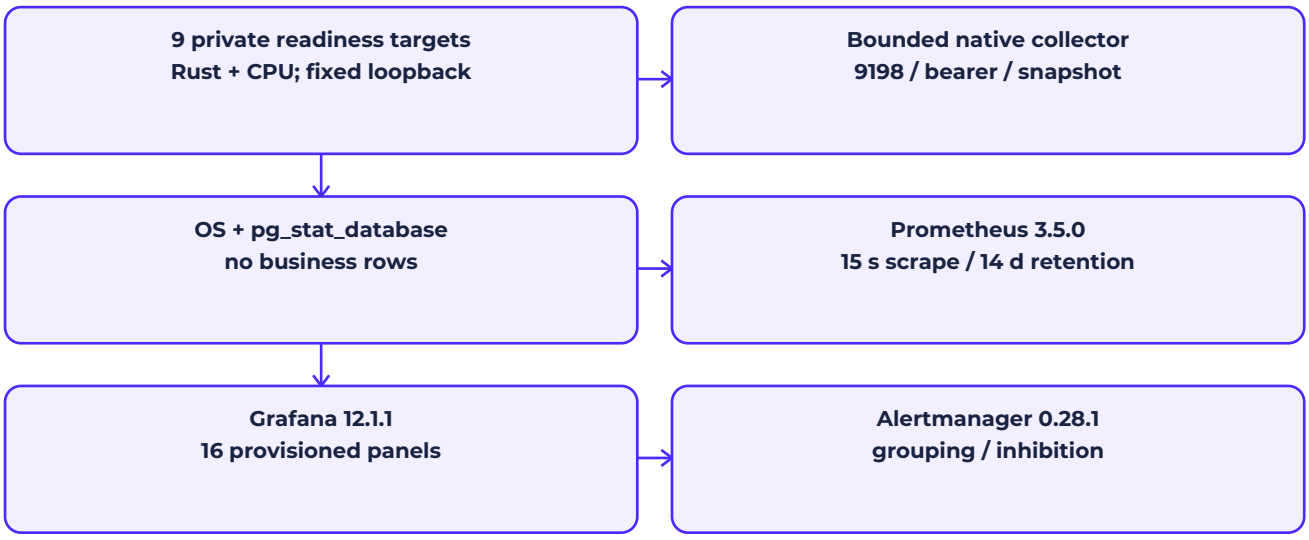
1. Эксплуатационная задача и границы наблюдения

Наблюдаемость Proofit отделена от бизнес-запроса: отказ системы мониторинга не останавливает поиск, аттестацию или расчёт сценария. Нативный сборщик читает фиксированные локальные readiness-endpoints и существующие CPU-счётчики, ресурсы ОС и агрегированную статистику PostgreSQL. Prometheus хранит временные ряды; Grafana визуализирует наблюдения; Alertmanager группирует операционные сигналы.

Контур обслуживает девять целей: gateway, семь Rust-доменов и CPU Intelligence. Проверка готовности Rust включает SELECT 1; CPU health подтверждает ответ процесса, но не качество модели. Отдельные OCR, Workbench, LLM, прогнозный процесс и Keycloak не включены в девять readiness-проб; их нельзя считать проверенными по зелёному статусу этой панели. Полные пользовательские сценарии остаются самостоятельным уровнем валидации.

2. Архитектура и потоки наблюдений

Сбор выполняется каждые 10 секунд; девять readiness-запросов запускаются с ограничением в девять потоков, тайм-аутом 3 секунды и пределом ответа 64 KiB. Используются фиксированные loopback-адреса, без перенаправлений и системного HTTP proxy. Снимок заменяется атомарно; scrape не инициирует дорогие SQL или probe-операции. Prometheus опрашивает сборщик каждые 15 секунд с тайм-аутом 5 секунд.



3. Каталог сигналов и измерительные контракты

Группа	Сигнал / источник	Корректная интерпретация
Доступность	proofit_service_up; 9 fixed labels	Ответ readiness, а не успешный найм или полная готовность AI
Readiness latency	proofit_health_probe_duration_seconds; cumulative histogram	P95 синтетических health-проб, не P95 пользовательского API
CPU-контур	operations_total, errors_total, cache_hits_total, processing_seconds_total	Счётчики из текущего процесса CPU; сброс при перезапуске корректно обрабатывается rate
Ресурсы	host CPU, available RAM, свободный диск; RSS и CPU seconds процессов	Хостовые показатели включают стороннюю нагрузку; процессные проверяют имя PID
PostgreSQL	соединения, commits, rollbacks, blks_hit/read, deadlocks	Агрегаты системной статистики; rollback не тождественен ошибке
Целостность	collector timestamp, duration, cpu_metrics_up, database_enabled/up	Доступность exporter не заменяет свежесть его данных

4. Формулы и различение смыслов

Ошибки вычислений = rate(errors) / max(rate(operations)+rate(errors)+rate(cache_hits), 0.001). Это доля в зарегистрированных CPU-исходах, а не доля всех HTTP-запросов: отказы авторизации и ранние отклонения входа могут не попасть в CPU counters. Суммарное processing time включает разные ветви; оно не объявляется точной средней задержкой выполнения задачи.

P95 readiness = histogram_quantile(0.95, sum by(service,le)(rate(probe_duration_bucket[5m]))). Границы histogram: 5/10/25/50/100/250/500/1000/2000/4000 ms и +Inf. Квантиль является оценкой по бакетам; низкий объём наблюдений ограничивает интерпретацию. При отсутствии данных панель показывает отсутствие данных, а не искусственный ноль.

PostgreSQL cache ratio = rate(blks_hit)/(rate(blks_hit)+rate(blks_read)), с безопасным нижним ограничением знаменателя. Это попадания shared buffers, не доля всех физических чтений диска. Использование соединений = все backend-процессы кластера / max_connections: знаменатель не сравнивается лишь с одной БД. Rate cumulative counters учитывает сброс процесса; gauge не дифференцируется.

5. Сигналы инцидента и операционные действия

В конфигурации 12 правил: два recording rules и десять alert rules. Пороги являются эксплуатационными стартовыми настройками, а не доказанным SLA. Внешняя доставка в почту или мессенджер не активируется без явной настройки получателя; firing-сигналы доступны в приватных UI Prometheus/Alertmanager.

Условие	Выдержка	Действие оператора
Exporter или service readiness отсутствует	1 минута	Listener, PID, Host-policy, DB; не отключать perimeter

Условие	Выдержка	Действие оператора
Снимок старше 45 секунд	1 минута	Проверить loop сборщика: scrape может работать при старом снимке
CPU counters недоступны	2 минуты	Схема и доступность фиксированного /metrics CPU
CPU error ratio >5%; минимум 20 исходов/5 мин	3 минуты	Ошибка вычислений, входной контракт, cache; не делать вывод из единичного запроса
Available RAM / free disk <10%	5 минут	Ресурсы процессов, конкурирующая нагрузка, политика retention
DB collector отказал / connections >80%	1 / 3 минуты	Выделенная роль, соединения, суммарный бюджет domain pools
Новый deadlock за 5 минут	Без дополнительной выдержки	Порядок транзакций и блокировки; не объявлять потерю данных

6. Безопасность, приватность и минимизация

Публичная маршрутизация Proofit не изменена. Prometheus: localhost:9095; Grafana: localhost:3001; Alertmanager: localhost:9096. Контейнерные порты опубликованы только на 127.0.0.1, сборщик по умолчанию слушает 127.0.0.1:9198. Его /metrics требует bearer-secret длиной не менее 32 символов; отсутствующий или неверный секрет возвращает 401. Grafana запрещает anonymous login и регистрацию; стартовый пароль генерируется независимо и хранится вне исходников.

Метрики не содержат email, идентификаторов кандидата, вакансии, резюме, SQL-текста или токенов. Единственная пользовательская размерность — фиксированное имя сервисной цели. Ошибки соединения не выводятся в exposition или журнал вместе с DSN. Нативные CPU-счётчики с некорректными, отрицательными или неполными значениями отклоняются; старый снимок counters не подставляется вместо текущего отказа.

Выделенная роль proofit_monitor имеет pg_read_all_stats без SELECT на users и resume_profiles, без superuser/createdb/createrole/replication. default_transaction_read_only=on; statement_timeout=2s; lock_timeout=1s. Клиент дополнительно включает read-only transaction и connect_timeout=3s. Статистическая роль может видеть расширенные системные сведения: exporter использует только фиксированные числовые SELECT. Перед production-подключением DBA проверяет отсутствие inherited/public-grants на бизнес-таблицы.

7. Ресурсные бюджеты и жизненный цикл

Prometheus: 512 MiB и 1 CPU, retention 14 дней с дополнительным ограничением 2 GB. Grafana: 384 MiB и 1 CPU. Alertmanager: 128 MiB. Данные хранятся в отдельных persistent volumes. Контейнеры имеют read-only root filesystem, cap_drop=ALL и no-new-privileges. Grafana использует временный /tmp и writable data volume. Версии образов закреплены; изменение версии требует проверки release notes и повторной проверки конфигураций.

Секреты не регенерируются при повторном prepare/start. Доступ с другой машины к Grafana или Prometheus намеренно не опубликован: для операторского доступа нужен отдельный защищённый административный канал. Grafana HTTP на loopback не объявляется публичным

защищённым endpoint; для удалённого размещения необходимы TLS, identity и ACL. Публичная ссылка продукта не предоставляет служебные панели.

8. Запуск и воспроизводимая проверка

Операция	Команда / место	Контроль
Зависимости	python -m pip install -r requirements.monitoring.txt	Pinned monitoring dependencies
Запуск	python -m scripts.start_monitoring	Collector без рестарта приложения; compose up -d
Остановка	python -m scripts.stop_monitoring	Только monitoring: volumes, роль и секреты сохраняются
Grafana	http://localhost:3001; user proofit-admin	Пароль: runtime/monitoring/grafana-password; не публиковать
Prometheus / alerts	localhost:9095 / localhost:9096	Targets=up; rules; active alerts
Unit	python -m pytest tests/test_monitoring.py	Token gate, schemas, stale omission, semantics
Live / browser	python -m scripts.check_monitoring_live	Access controls, queries, DB least privilege, Grafana
Правила	promtool check config; check rules; test rules	deploy/monitoring/rules.test.yml
PDF	python -m scripts.build_monitoring_documentation	Обновляет только настоящий документ

9. Фактическая приёмка текущего контура

Измерение: 2026-10-11T10:51:32+0300. Пройдены 9 unit-тестов и 16 live-проверок без участия LLM. Prometheus принимает 28 metric families / 170 samples; девять readiness-целей доступны. Все 16 PromQL выражений приняты API; provisioned dashboard открыт в настоящем браузере. Время конкретного цикла сбора: 0.462 s; это один измеренный цикл, не P95 и не benchmark платформы.

Promtool подтвердил синтаксис Prometheus-конфигурации и 12 правил. Тестовые ряды отдельно проверили срабатывание ExporterDown и ServiceDown после выдержки, а также отсутствие DatabaseDown при выключенном DB collector. Секретный доступ 200 и два отказа 401 проверены live. Выделенная статистическая роль действительно read-only и не читает users/resume_profiles. Протоколы: monitoring-unit.xml, monitoring-live.json; визуальный артефакт: monitoring-grafana.png.

Сбор наблюдений не является нагрузочным тестом и не подтверждает снижение времени найма. Доступность, ресурсный бюджет и качество бизнес-прогноза — разные измерительные задачи. Интеграция не требует изменения графа, Horizon или логики квалификации.

10. Первичные источники и привязка к реализации

Prometheus: Instrumentation — типы метрик, ограничение labels, различение counters и gauges, контроль свежести наблюдений.

Prometheus: Configuration — scrape, credentials_file, recording/alert rules и Alertmanager.

Grafana: Provisioning — воспроизводимое управление datasource и dashboards. Первичные документы проверены 10 октября 2026 года; они обосновывают механизм, а не подтверждают независимый production SLA Proofit.

Реализация: scripts/monitoring_exporter.py; start_monitoring.py; prepare_monitoring.py; deploy/monitoring/. Evidence manifest: validation/reports/current/monitoring-source.json. SHA-256 среза monitoring source: 3c91c0e6fe2275e51663ad6585def81859a07170c5ac8fd79eb09e08a1b292a4.